

PROGETTO UNO FLIP!

Manuale Tecnico, Architettura e Diario dei Bug

LA BIBBIA DI UNO FLIP! (C++)

Manuale Tecnico Completo, Guida di Studio e Diario di Sviluppo

INTRODUZIONE AL SISTEMA

Il nostro progetto "UNO Flip!" in C++ è molto più di un semplice gioco da tavolo digitale. È un ecosistema complesso suddiviso rigorosamente secondo il **Pattern MVC (Model-View-Controller)** e lo **State Pattern**. Questo garantisce che la logica del gioco (che calcola i turni e le regole) non sappia assolutamente nulla di come i bottoni vengano disegnati a schermo o di come i pacchetti viaggino su internet.

Come funziona interamente (Flusso Logico Globale)

- 1. L'Avvio (main.cpp e Main_grafico.cpp):** Il `main()` istanzia la finestra `sf::RenderWindow` (SFML) e inizializza il **SceneManager**. Da questo momento, il ciclo `while (window.isOpen())` prende il controllo, calcolando il tempo trascorso (`deltaTime`) e smistando Eventi, Aggiornamenti grafici e Rendering alla "Scena" attualmente attiva.
- 2. Navigazione (State Pattern):** Ogni schermata (`LoginScene`, `MenuScene`, `GameScene`, ecc.) eredita da `Scene`. Quando vogliamo cambiare menu, chiamiamo `manager->changeScene(std::make_unique<NuovaScena>())`.
- 3. La Partita Inizia (GameScene + Partita):** Quando si avvia il gioco, `GameScene` istanzia un oggetto di classe `Partita`.
- 4. GameScene (Vista/Controller)** intercetta i click del mouse.
- 5. Traduce i click** (es. "L'utente ha cliccato la terza carta") in chiamate logiche (`partita.mossaUmano(2, -1, false)`).
- 6. Partita (Modello)** elabora la regola, controlla se è valida, cambia il turno e aggiorna lo stato interno.
- 7. Al frame successivo**, `GameScene` legge il nuovo stato (chi è il giocatore di turno, qual è la carta in cima) e lo ridisegna a schermo.

- 8. Il Multiplayer (Rete):** Se il gioco è multiplayer, il click del mouse non genera solo la mossa locale, ma formatta una stringa (`MOSSA;2;0;0`) e la invia tramite **Socket TCP non bloccanti** al computer avversario, che esegue `mossaRete( . . . )` replicando la logica esattamente in simultanea.

ANALISI DI OGNI SINGOLO FILE

In questa sezione analizziamo a fondo ogni file sorgente, spiegandone il vero scopo all'interno della macchina del progetto.

1. IL MODELLO (La Logica Pura)

Carta.h e carta.cpp - **Scopo:** Rappresenta una singola carta fisica con due facce. - **Funzionamento:** A differenza dell'UNO classico, qui la carta ha attributi per il "Lato Chiaro" (Colori base, carte normali) e "Lato Oscuro" (Colori strani, effetti devastanti come +5 o Salta Tutti). Memorizza *entrambi* i lati alla nascita tramite il costruttore. - **Metodi chiave:** `getColore(bool latoOscuro)` e `getValore(bool latoOscuro)`. A seconda di quale lato del gioco è attualmente attivo globalmente, la carta "mente" su se stessa e rivela solo le statistiche della faccia rivolta verso l'alto.

Mazzo.h e Mazzo.cpp - **Scopo:** Fabbrica di carte e gestore delle pile (mazzo di pesca e pila degli scarti). - **Funzionamento:** - `inizializzaMazzo()` genera 112 carte. Crea vettori per tutte le combinazioni di lati chiari e oscuri, le randomizza e le incolla assieme creando le carte doppie. - Usa una matematica basata sul **Generatore Congruenziale Lineare (LCG)** per assicurare che il mescolamento sia universale (vedi capitolo dei Bug per dettagli!). - **Metodi chiave:** `pesca()` (prende l'ultimo elemento e lo cancella dal mazzo), `scarta(Carta c)`.

Giocatore.h e Giocatore.cpp - **Scopo:** L'entità al tavolo da gioco. - **Funzionamento:** Mantiene semplicemente un nome testuale, un flag booleano `isBot` (per sapere se le sue mani devono essere guidate dal computer) e il vettore dinamico `std::vector<Carta> mano`.

Partita.h e Partita.cpp - **Scopo:** L'arbitro supremo e cuore pulsante della logica. - **Funzionamento:** Mantiene l'indice del turnoCorrente, la direzione sensoOrario (booleano invertito dai cambia-giro), e il booleano fondamentale `latoOscuroAttivo` (invertito dalla carta FLIP). - **Controllo Regole (mossaValida):** Verifica matematicamente se Colore e Valore della carta giocata combaciano col tavolo (o se è nera, Jolly). - **Applica Effetto (applicaEffettoCarta):** Enorme smistatore if-else. Gestisce chi pesca 1, chi pesca 5, chi inverte, e chi decide i colori. - **Le Mosse:** Espone `mossaUmano`, `mossaBot` e `mossaRete`.

Database.h e Database.cpp - **Scopo:** Leggere e salvare le statistiche generali dei giocatori a vita (Vittorie, Partite Giocate). - **Funzionamento:** All'avvio, scarica il file `classifica.csv` nella RAM caricando un `std::vector<Statistiche> records`. A fine partita, aggiorna il vettore e riscrive brutalmente il file sovrascrivendolo, mantenendo la persistenza in formato testo.

2. LA VISTA E IL CONTROLLER (Grafica SFML 3)

Scene.h e SceneManager.h/.cpp - **Il Motore della Macchina a Stati.** Scene definisce i 3 metodi obbligatori: `handleInput()`, `update()`, `render()`. - SceneManager mantiene i font e le texture globali (background) per non caricarli 100 volte appesantendo la RAM. Cambia la schermata distruggendo il vecchio oggetto e creando il nuovo.

Le Scene Grafiche (Schermate) - LoginScene: Gestisce gli eventi tastiera ASCII per permettere all'utente di digitare il proprio nome. - **MenuScene:** Disegna i pulsanti e reindirizza verso singleplayer, host o join. - **LobbyScene:** Avvia un Server (in attesa) o un Client (che tenta di connettersi). Una volta

connessi, avviene il cruciale *Handshake* del Seme Random. - **GameScene (Il file più complesso del gioco!)**: - Calcola la coordinata X/Y di ogni singola carta in mano in modo responsivo (più carte hai, più si sovrappongono per stare nello schermo). - Gestisce l'interfaccia utente (pulsanti "UNO", "PESCA", Pop-up scelta colore). - Interroga `GestoreRete` in ogni frame per ricevere pacchetti dall'avversario. - Alimenta il motore Audio in base all'azione eseguita.

Utilità Grafiche e Sonore - `UIUtils.h/.cpp`: Fabbrica di Bottoni (metodo `disegnaPulsante`) che include logica di Hover (se il mouse ci passa sopra diventa più chiaro/scuro). -

`SoundManager.h/.cpp`: Precarica i buffer audio .wav al volo e istanzia degli `sf::Sound` (effetti di carte shufflate, vittoria, click di errore).

3. LA RETE (Network Multiplayer)

`GestoreRete.h` e `GestoreRete.cpp` - Implementano le classi `ServerRete` (Host) e `ClientRete` (Guest). - **IL SEGRETO**: I Socket TCP di base freezano l'applicazione in attesa di dati. Nel nostro progetto chiamiamo `socket.setBlocking(false)`. Questo costringe SFML a controllare il socket, e se non c'è nulla ritorna un errore specifico (not ready), permettendoci di bypassare l'attesa e disegnare i 60 FPS dello schermo in santa pace. Il multiplayer diventa così totalmente fluido (modello *Lockstep* o Polling Asincrono).

DEEP DIVE: COME FUNZIONA L'INTELLIGENZA ARTIFICIALE (AI / BOT)

Il nostro bot (Intelligenza Artificiale C++) è governato all'interno della funzione

`Partita::mossaBot()`. Non è un semplice generatore casuale, ma esegue valutazioni basate su regole di priorità (un classico approccio *Greedy Algorithm*).

1. **Simulazione Umana ("UNO!")**:
2. La prima cosa che fa l'AI è accorgersi se ha in mano 2 carte. Se gioca una carta rimarrà con 1 carta sola. Prima ancora di giocarla, il sistema fa emettere al bot un log a schermo in cui dichiara virtualmente "UNO!".
3. **Ricerca dell'Attacco Perfetto**:
4. Il bot cicla tutte le carte valide (`mossaValida(mano[i])`).
5. Cerca esplicitamente carte aggressive: `PESCA_CINQUE`, `SALTA_TUTTI`, o `JOLLY_PESCA_COLORE`. Se ne trova una, seleziona immediatamente quella, preferendola a qualsiasi carta normale.
6. **Mossa Standard o Pesca**:
7. Se non ha carte aggressive, cicla di nuovo per cercare semplicemente la prima carta valida disponibile.
8. Se ha trovato una mossa, la gioca. Nel caso in cui getti un Jolly (che richiede la scelta di un colore), seleziona il colore avvantaggiato usando la formula modulo 4.
9. Se non ha mosse valide, chiama `bot.pescaCarta(mazzo.pesca())` e subisce il turno a vuoto.

Tutto questo processo avviene in una frazione di millisecondo, dopodiché il controller grafico emette i suoni e aggiorna la schermata.

IL DIARIO DEI BUG: COME LI ABBIAMO TROVATI E

DISTRUTTI

BUG #1: IL PARADOSSO DI MAC VS WINDOWS (Desincronizzazione di Rete)

- **Come l'abbiamo scoperto:** Facendo giocare in LAN un computer macOS Apple Silicon con un PC Windows, i giocatori si connettevano ma finivano con mani di carte totalmente diverse! La carta in cima agli scarti su Mac era un 3 Rosso, su Windows era un +5 Oscuro. Un incubo.
- **Come ci siamo arrivati (L'Indagine):** Ci siamo accorti che, nonostante si passassero un "seme random" iniziale (`unsigned seed = ...`), le funzioni della libreria standard `std::shuffle` usano algoritmi sotto il cofano (`std::default_random_engine`) che sono implementati diversamente dal compilatore *Clang* di Apple rispetto a *MSVC* di Microsoft. Pur partendo dallo stesso numero (es. 12345), i due computer mischiavano il mazzo in ordini completamente alieni l'uno all'altro.
- **LA SOLUZIONE:** Abbiamo bandito le funzioni casuali di C++! Abbiamo scritto un nostro generatore matematico interno (il **Generatore Congruenziale Lineare - LCG**) in `Mazzo::ottieniNumeroCasuale().cpp` `seed_rete = seed_rete * 1664525 + 1013904223; return seed_rete;` La matematica pura non cambia mai, indipendentemente dal Sistema Operativo. Ora, Mac e Windows generano byte per byte la stessa identica sequenza, mantenendo il tavolo sincronizzato al 100%.

BUG #2: IL BUG DELLA "PESCA INFINITA"

- **Come l'abbiamo scoperto:** Un giocatore, per testare il sistema o per puro fastidio, iniziava a cliccare violentemente sul mazzo di pesca 50 volte in un secondo. Il gioco continuava a dargli carte, riempiendo la sua mano all'infinito e senza mai passare il turno all'avversario.
- **L'Indagine:** All'interno del codice di `mossaUmano`, quando `indiceCarta` era -1 (che indica il click sul mazzo), il giocatore pescava la carta ma l'algoritmo usciva brutalmente dalla funzione `return;` senza chiamare `passaTurno()`.
- **LA SOLUZIONE:** Abbiamo modificato le righe 114 in `Partita.cpp`. Ora, ogni volta che la variabile intercetta una pescata (`indiceCarta == -1`), prima del `return` viene *obbligatoriamente* chiamato il metodo `passaTurno()`. Il bug è stato risolto e il flusso della partita messo in sicurezza.

BUG #3: IL BLOCCO DEL "JOLLY GRIGIO" DURANTE UN FLIP

- **Come l'abbiamo scoperto:** Durante una partita intensa, un utente giocava una carta **FLIP** per girare il mondo nel Lato Oscuro. Se, in pura casualità, sul fondo del mazzo scarti (che in quel momento era "sotto") c'era stata giocata in precedenza una carta **JOLLY** (di colore base NERO), il gioco crashava o rimaneva freezato in un colore illegale, impedendo a chiunque di scartare carte.
- **L'Indagine:** La carta FLIP attiva il comando `latoOscuroAttivo = !latoOscuroAttivo`, poi guarda la carta in cima agli scarti e ne assume il colore base come "Colore Attivo" del tavolo. Se quella carta si rivela essere un NERO (Jolly), il `coloreAttivo` diventava NERO. NESSUNA carta giocabile (eccetto un altro jolly) può essere appoggiata sul nero, bloccando permanentemente il tavolo perché ai Jolly non era stato assegnato il pop-up di ricalcolo postumo!
- **LA SOLUZIONE:** Abbiamo introdotto l'intelligenza di ripiego nella funzione del Flip (riga 73 di `Partita.cpp`). Se il Flip rivela un Jolly nero nascosto dietro la pila: `cpp if (nuovoColore == NERO) { int r = Mazzo::ottieniNumeroCasuale() % 4; // Forza a caso uno dei 4 colori primari del lato corretto, // per riavviare la giocabilità. }` In questo modo il gioco si sblocca autonomamente assegnando magicamente un colore randomico

coerente col lato attuale.

Fine del Documento. Sezione creata interamente in via procedurale a scopo di ripasso.